

IFVG Sniper Model — Pine Script v6

TradingView indicator · defaults tuned for the 15-minute chart · 643 lines. Paste into Pine Editor → Save → Add to chart.

```
1 //@version=6
2 //
3 // IFVG Sniper Model (defaults tuned for the 15-minute chart)
4 // 1. Liquidity sweep — price wicks through a swing / previous-day high-low and closes back inside
5 // 2. V-shape recovery — fast displacement away from the sweep extreme (× ATR)
6 // 3. CISD — close through the open of the delivery series that made the extreme
7 // 4. IFVG — an opposing FVG is closed through by a displacement candle → sniper zone
8 // 5. Retest + sweep — price returns into the IFVG, sweeps minor liquidity and holds → entry
9 // 6. Targets — fixed 1R / 2R (default) with a draw on liquidity required beyond TP1
10 // Filters — higher-timeframe bias, London / New York killzones, max stop size,
11 // discount / premium entry, daily trade & loss limits, one position at a time
12 //
13 indicator("IFVG Sniper Model", "IFVG Sniper", overlay = true, max_bars_back = 500,
14 max_lines_count = 500, max_labels_count = 500, max_boxes_count = 500)
15
16 // ————— Inputs —————
17 const string G0 = "0 · High-Probability Filters"
18 useHtf = input.bool(true, "Trade with higher-timeframe bias", group = G0,
19 tooltip = "Longs only when the HTF close is above a rising EMA, shorts only below a falling EMA
20 (last closed HTF bar, no repaint).")
21 htfTf = input.timeframe("60", "HTF", group = G0, inline = "htf")
22 htfEmaLen = input.int(50, "EMA", minval = 1, group = G0, inline = "htf")
23 useKz = input.bool(true, "Entries only inside killzones", group = G0)
24 kz1 = input.session("0200-0500", "London", group = G0, inline = "kz")
25 kz2 = input.session("0700-1100", "New York", group = G0, inline = "kz")
26 kzTz = input.string("America/New_York", "Killzone timezone", group = G0)
27 dispBody = input.float(0.5, "Min body % of IFVG inversion candle", minval = 0, maxval = 1, step = 0.05,
28 group = G0,
29 tooltip = "The candle that closes through the FVG must be a displacement candle (body / range). 0
30 disables.")
31 pdMin = input.float(0.5, "Min retracement into the V leg", minval = 0, maxval = 1, step = 0.05, group
32 = G0,
33 tooltip = "Entry must sit at least this deep in the range from the sweep extreme to the V high
34 (bull) / low (bear). 0.5 = discount for longs, premium for shorts. 0 disables.")
35 maxDay = input.int(2, "Max trades per day", minval = 1, group = G0, inline = "day")
36 maxLossDay = input.int(1, "Stop after N losses", minval = 1, group = G0, inline = "day")
37 maxSlAtr = input.float(2.0, "Skip entry if stop > N × ATR", minval = 0.2, step = 0.1, group = G0)
38 minSlAtr = input.float(0.3, "Minimum stop size (× ATR)", minval = 0, step = 0.05, group = G0,
39 tooltip = "Stops tighter than this get widened so normal noise does not tag them.")
40
41 const string G1 = "1 · Liquidity Sweep"
42 swingLen = input.int(8, "Liquidity swing length", minval = 2, group = G1,
43 tooltip = "Bars on each side that define a swing high/low (a resting liquidity pool). Higher = only
44 major liquidity.")
45 usePdl = input.bool(true, "Add previous-day high / low as liquidity", group = G1)
46 useAsia = input.bool(true, "Add Asia session high / low as liquidity", group = G1, inline = "asia")
47 asiaSess = input.session("2000-0000", "", group = G1, inline = "asia")
48 maxPools = input.int(30, "Max pools tracked per side", minval = 5, maxval = 100, group = G1)
49 showPools = input.bool(true, "Show resting liquidity levels", group = G1)
50
51 const string G2 = "2 · V-Shape Recovery"
52 vMaxBars = input.int(6, "Max bars from extreme → recovery", minval = 1, group = G2)
53 vAtrMult = input.float(1.5, "Recovery displacement (× ATR)", minval = 0.1, step = 0.1, group = G2)
54 atrLen = input.int(14, "ATR length", minval = 1, group = G2)
55
56 const string G3 = "3 · CISD + IFVG"
57 alignBars = input.int(12, "Max bars sweep → CISD & IFVG", minval = 1, group = G3)
58 cisdScan = input.int(15, "CISD candle scan depth", minval = 1, maxval = 100, group = G3)
59 fvgLook = input.int(20, "FVG may form up to N bars before sweep", minval = 1, group = G3)
60 fvgMinAtr = input.float(0.1, "Min FVG size (× ATR)", minval = 0, step = 0.05, group = G3)
61
62 const string G4 = "4 · Retest + Second Sweep"
63 retestMax = input.int(16, "Max bars to wait for retest", minval = 1, group = G4)
64 minorLen = input.int(3, "Retest sweep lookback (bars)", minval = 1, group = G4,
65 tooltip = "On the retest, price must take the low (bull) / high (bear) of the previous N bars and
66 close back.")
67
68 const string G5 = "5 · Targets & Management"
```

```

62 tpMode      = input.string("Fixed R:R", "Target mode", options = ["Fixed R:R", "Liquidity (capped at
R:R)"], group = G5,
63     tooltip = "Fixed R:R: TP1 / TP2 exactly at the R multiples below.\nLiquidity (capped): nearest draw
on liquidity, never further than the R multiples.")
64 rr1         = input.float(1.0, "TP1 (R)", minval = 0.25, step = 0.25, group = G5, inline = "rr")
65 rr2         = input.float(2.0, "TP2 (R)", minval = 0.5, step = 0.25, group = G5, inline = "rr")
66 requireDol  = input.bool(true, "Require draw on liquidity beyond TP1", group = G5,
67     tooltip = "Only enter when an untouched liquidity pool sits at least TP1 away in the trade
direction.")
68 slBuf       = input.float(0.1, "SL buffer (× ATR)", minval = 0, step = 0.05, group = G5)
69 partialPct  = input.float(50, "Close % at TP1", minval = 0, maxval = 100, step = 5, group = G5)
70 beAfterTp1  = input.bool(true, "Move SL to entry after TP1", group = G5)
71 tradeMax    = input.int(96, "Close trade after N bars", minval = 1, group = G5)
72
73 const string G6 = "Display"
74 showBull    = input.bool(true, "Bullish setups", group = G6, inline = "d")
75 showBear    = input.bool(true, "Bearish setups", group = G6, inline = "d")
76 bullCol     = input.color(#089981, "Bull", group = G6, inline = "c")
77 bearCol     = input.color(#F23645, "Bear", group = G6, inline = "c")
78 tgtCol      = input.color(#2962FF, "Targets", group = G6, inline = "c")
79 slCol       = input.color(#FF9800, "Stop", group = G6, inline = "c")
80 showStats   = input.bool(true, "Show stats table", group = G6)
81
82 // ----- Core series -----
83 atr         = ta.atr(atrLen)
84 minorLow    = ta.lowest(low, minorLen)[1]
85 minorHigh   = ta.highest(high, minorLen)[1]
86 ph         = ta.pivohigh(high, swingLen, swingLen)
87 pl         = ta.pivotlow(low, swingLen, swingLen)
88
89 // Higher-timeframe bias from the last CLOSED HTF bar (lookahead + [1] = no repaint)
90 htfData() =>
91     float e = ta.ema(close, htfEmaLen)
92     [close[1], e[1], e[2]]
93 [htfClose, htfEma, htfEmaPrev] = request.security(syminfo.tickerid, htfTf, htfData(), lookahead =
barmerge.lookahead_on)
94 bool bullBias = not useHtf or (htfClose > htfEma and htfEma > htfEmaPrev)
95 bool bearBias = not useHtf or (htfClose < htfEma and htfEma < htfEmaPrev)
96
97 // Killzones
98 bool inKz = not useKz or not na(time(timeframe.period, kz1, kzTz)) or not na(time(timeframe.period, kz2,
kzTz))
99
100 // Displacement candle (body share of range) for the IFVG inversion
101 float rng   = high - low
102 bool bullDisp = close > open and rng > 0 and (close - open) / rng >= dispBody
103 bool bearDisp = close < open and rng > 0 and (open - close) / rng >= dispBody
104
105 // ----- Liquidity pools -----
106 var array<float> bslPx = array.new<float>() // buy-side liquidity (swing highs)
107 var array<int> bslBar = array.new<int>()
108 var array<line> bslLn = array.new<line>()
109 var array<float> sslPx = array.new<float>() // sell-side liquidity (swing lows)
110 var array<int> sslBar = array.new<int>()
111 var array<line> sslLn = array.new<line>()
112
113 addPool(array<float> px, array<int> bx, array<line> ln, float p, int b) =>
114     line l = na
115     if showPools
116         l := line.new(b, p, bar_index, p, color = color.new(color.gray, 40), style = line.style_dotted,
extend = extend.right)
117     px.push(p)
118     bx.push(b)
119     ln.push(l)
120     if px.size() > maxPools
121         px.shift()
122         bx.shift()
123         line.delete(ln.shift())
124
125 // Removes every pool traded through this bar. Returns the most extreme pool that was
126 // swept AND reclaimed (wick through, close back) – d = 1 checks lows, d = -1 checks highs.
127 scanSweeps(array<float> px, array<int> bx, array<line> ln, int d) =>
128     float lvl = na
129     int lb = na

```

```

130     int    n    = px.size()
131     if n > 0
132         for i = n - 1 to 0
133             float p = px.get(i)
134             bool taken = d == 1 ? low < p : high > p
135             if taken
136                 bool reclaimed = d == 1 ? close > p : close < p
137                 if reclaimed and (na(lvl) or d * (lvl - p) > 0)
138                     lvl := p
139                     lb := bx.get(i)
140                     line l = ln.get(i)
141                     if not na(l)
142                         line.set_x2(l, bar_index)
143                         line.set_extend(l, extend.none)
144                     px.remove(i)
145                     bx.remove(i)
146                     ln.remove(i)
147     [lvl, lb]
148
149 // ----- Fair value gaps -----
150 var array<float> bearFvgTop = array.new<float>() // bearish FVGs → become bullish IFVGs
151 var array<float> bearFvgBot = array.new<float>()
152 var array<int>   bearFvgBar = array.new<int>()
153 var array<float> bullFvgTop = array.new<float>() // bullish FVGs → become bearish IFVGs
154 var array<float> bullFvgBot = array.new<float>()
155 var array<int>   bullFvgBar = array.new<int>()
156
157 addFvg(array<float> top, array<float> bot, array<int> bx, float t, float b, int x) =>
158     top.push(t)
159     bot.push(b)
160     bx.push(x)
161     if top.size() > 100
162         top.shift()
163         bot.shift()
164         bx.shift()
165
166 // Removes every FVG closed through this bar. Returns the most recent inverted FVG that
167 // formed at/after minBar. d = 1 → bearish FVG closed above, d = -1 → bullish FVG closed below.
168 scanInversions(array<float> top, array<float> bot, array<int> bx, int d, int minBar) =>
169     float zt = na
170     float zb = na
171     int   zbar = na
172     int   n = top.size()
173     if n > 0
174         for i = n - 1 to 0
175             bool inverted = d == 1 ? close > top.get(i) : close < bot.get(i)
176             if inverted
177                 int fb = bx.get(i)
178                 if fb >= minBar and (na(zbar) or fb > zbar)
179                     zt := top.get(i)
180                     zb := bot.get(i)
181                     zbar := fb
182                 top.remove(i)
183                 bot.remove(i)
184                 bx.remove(i)
185     [zt, zb, zbar]
186
187 // CISD reference: open of the earliest candle in the consecutive opposing series that
188 // delivered price into the current extreme (bull: down-close candles, bear: up-close).
189 cisdLevel(int d) =>
190     float lvl = na
191     int   lb = na
192     bool run = false
193     for i = 0 to cisdScan
194         bool opp = d == 1 ? close[i] < open[i] : close[i] > open[i]
195         if opp
196             lvl := open[i]
197             lb := bar_index - i
198             run := true
199         else if run
200             break
201     [lvl, lb]
202
203 // Draw on liquidity: nearest two untouched pools in trade direction, at least minDist away.

```

```

204 findTargets(int d, float entry, float minDist) =>
205     array<float> pools = d == 1 ? bs1Px : sslPx
206     float t1 = na
207     float t2 = na
208     for p in pools
209         if d * (p - entry) >= minDist
210             if na(t1) or d * (t1 - p) > 0
211                 t2 := t1
212                 t1 := p
213             else if p != t1 and (na(t2) or d * (t2 - p) > 0)
214                 t2 := p
215     [t1, t2]
216
217 // ----- Setup state machine -----
218 // st: 0 idle · 1 swept (waiting V + CISD + IFVG) · 2 IFVG armed (waiting retest) · 3 in trade
219 type Setup
220     int    dir
221     int    st      = 0
222     float  lvl
223     int    lvlBar
224     int    sweepBar
225     float  ext
226     int    extBar
227     bool   vDone    = false
228     int    vBar
229     float  vPx
230     float  cisd
231     int    cisdBar
232     bool   cisdDone = false
233     int    cisdConfBar
234     bool   zDone    = false
235     float  zTop
236     float  zBot
237     int    zBar
238     int    armBar
239     float  far      // V-leg extreme in trade direction (for discount / premium)
240     float  entry
241     float  sl
242     float  tp1
243     float  tp2
244     float  risk
245     float  r1
246     float  r2
247     bool   tp1Hit   = false
248     int    entryBar
249     box    zBox
250     line   lE
251     line   lS
252     line   lT1
253     line   lT2
254     label  bS
255     label  bT1
256     label  bT2
257
258 method reset(Setup s) =>
259     s.st      := 0
260     s.vDone   := false
261     s.cisdDone := false
262     s.zDone   := false
263     s.tp1Hit  := false
264     s.zBox    := na
265     s.lE      := na
266     s.lS      := na
267     s.lT1     := na
268     s.lT2     := na
269     s.bS      := na
270     s.bT1     := na
271     s.bT2     := na
272     true
273
274 method start(Setup s, float lvl, int lvlBar) =>
275     float adv = s.dir == 1 ? low : high
276     if s.st == 0
277         [c, cb] = cisdLevel(s.dir)

```

```

278     s.st      := 1
279     s.lvl     := lvl
280     s.lvlBar  := lvlBar
281     s.sweepBar := bar_index
282     s.ext     := adv
283     s.extBar  := bar_index
284     s.cisd    := c
285     s.cisdBar := cb
286     s.far     := s.dir == 1 ? high : low
287 else if s.st == 1 and s.dir * (s.lvl - lvl) > 0
288     s.lvl     := lvl
289     s.lvlBar  := lvlBar
290     true
291
292 method setTradeX(Setup s, int x) =>
293     line.set_x2(s.lE, x)
294     line.set_x2(s.lS, x)
295     line.set_x2(s.lT1, x)
296     line.set_x2(s.lT2, x)
297     label.set_x(s.bS, x)
298     label.set_x(s.bT1, x)
299     label.set_x(s.bT2, x)
300     true
301
302 // bias: HTF bias agrees with this direction · allow: killzone, daily limits and no open opposite trade
303 method update(Setup s, float atrV, float minorLvl, float zt, float zb, int zbar, bool bias, bool allow)
304 =>
305     int d = s.dir
306     color c = d == 1 ? bullCol : bearCol
307     float adv = d == 1 ? low : high
308     float fav = d == 1 ? high : low
309     bool evArm = false
310     bool evEntry = false
311     bool evTp1 = false
312     bool evTp2 = false
313     bool evSl = false
314     bool evBe = false
315     float resR = na // realised R when the trade closes on this bar
316     float part = partialPct / 100
317
318     if s.st == 1
319         // New extreme → any partial confirmation is void, rebuild from the new low/high
320         if d * (s.ext - adv) > 0
321             [cl, cb] = cisdLevel(d)
322             s.ext := adv
323             s.extBar := bar_index
324             s.vDone := false
325             s.cisdDone := false
326             s.zDone := false
327             s.cisd := cl
328             s.cisdBar := cb
329             s.far := fav
330             s.far := na(s.far) or d * (fav - s.far) > 0 ? fav : s.far
331             if not na(zbar)
332                 s.zTop := zt
333                 s.zBot := zb
334                 s.zBar := zbar
335                 s.zDone := true
336             if not s.vDone and bar_index - s.extBar <= vMaxBars and d * (fav - s.ext) >= vAtrMult * atrV and
337 d * (close - s.lvl) > 0
338                 s.vDone := true
339                 s.vBar := bar_index
340                 s.vPx := fav
341             if not s.cisdDone and not na(s.cisd) and d * (close - s.cisd) > 0
342                 s.cisdDone := true
343                 s.cisdConfBar := bar_index
344
345         if s.vDone and s.cisdDone and s.zDone
346             s.st := 2
347             s.armBar := bar_index
348             evArm := true
349             line.new(s.lvlBar, s.lvl, s.sweepBar, s.lvl, color = c, style = line.style_dashed)
350             label.new(s.extBar, s.ext, d == 1 ? "SSL sweep" : "BSL sweep", color = color(na), textcolor =

```

c,

```

349         style = d == 1 ? label.style_label_up : label.style_label_down, size = size.small)
350     label.new(s.vBar, s.vPx, "V", color = color(na), textcolor = c,
351         style = d == 1 ? label.style_label_down : label.style_label_up, size = size.small)
352     line.new(s.cisdBar, s.cisd, s.cisdConfBar, s.cisd, color = c, width = 2)
353     label.new(s.cisdConfBar, s.cisd, "CISD", color = color(na), textcolor = c,
354         style = label.style_label_left, size = size.small)
355     s.zBox := box.new(s.zBar, s.zTop, bar_index, s.zBot, border_color = c, bgcolor = color.new(c,
80),
356         text = "⚡ IFVG", text_color = c, text_size = size.small, text_halign = text.align_right)
357     else if bar_index - s.sweepBar > alignBars or (not s.vDone and bar_index - s.extBar > vMaxBars)
358         s.reset()
359
360     else if s.st == 2
361         float zNear = d == 1 ? s.zTop : s.zBot
362         float zFar = d == 1 ? s.zBot : s.zTop
363         box.set_right(s.zBox, bar_index)
364         s.far := d * (fav - s.far) > 0 ? fav : s.far
365         if bar_index > s.armBar
366             bool broke = d * (close - zFar) < 0 or d * (adv - s.ext) < 0
367             bool expired = bar_index - s.armBar > retestMax
368             bool touched = d * (adv - zNear) <= 0
369             bool sweep2 = d * (adv - minorLvl) < 0 and d * (close - minorLvl) > 0
370             bool holds = d * (close - zFar) >= 0
371             if broke or expired
372                 box.set_bgcolor(s.zBox, color.new(color.gray, 85))
373                 box.set_border_color(s.zBox, color.gray)
374                 box.set_text_color(s.zBox, color.gray)
375                 s.reset()
376             else if touched and sweep2 and holds and bias and allow
377                 float entry = close
378                 float legSz = math.abs(s.far - s.ext)
379                 float depth = legSz > 0 ? d * (s.far - entry) / legSz : 0.0
380                 float risk = math.max(math.abs(entry - (adv - d * slBuf * atrV)), minSlAtr * atrV)
381                 [dol, dol2] = findTargets(d, entry, rr1 * risk)
382                 bool okSize = risk <= maxSlAtr * atrV
383                 bool okDol = not requireDol or not na(dol)
384                 bool okPd = depth >= pdMin
385                 // A retest that fails the stop-size / DOL / discount filters is skipped; the zone stays
armed.
386                 if okSize and okDol and okPd
387                     s.st := 3
388                     s.entryBar := bar_index
389                     s.entry := entry
390                     s.risk := risk
391                     s.sl := entry - d * risk
392                     float cap1 = entry + d * rr1 * risk
393                     float cap2 = entry + d * rr2 * risk
394                     if tpMode == "Fixed R:R"
395                         s.tp1 := cap1
396                         s.tp2 := cap2
397                     else
398                         [t1, t2] = findTargets(d, entry, 0.5 * risk)
399                         s.tp1 := na(t1) or d * (t1 - cap1) > 0 ? cap1 : t1
400                         s.tp2 := na(t2) or d * (t2 - cap2) > 0 ? cap2 : t2
401                         if d * (s.tp2 - s.tp1) <= 0
402                             s.tp2 := entry + d * math.max(rr2 * risk, math.abs(s.tp1 - entry) + 0.5 *
risk)
403
404                     s.r1 := math.abs(s.tp1 - entry) / risk
405                     s.r2 := math.abs(s.tp2 - entry) / risk
406                     evEntry := true
407
408                     int x2 = bar_index + 5
409                     string tip = "Entry " + str.tostring(s.entry, format.mintick) +
410                         "\nSL " + str.tostring(s.sl, format.mintick) +
411                         "\nTP1 " + str.tostring(s.tp1, format.mintick) + " (" + str.tostring(s.r1,
412                             "#.##") + "R)" +
413                         "\nTP2 " + str.tostring(s.tp2, format.mintick) + " (" + str.tostring(s.r2,
414                             "#.##") + "R)" +
415                         "\nDraw on liquidity " + (na(dol) ? "none beyond TP1" : str.tostring(dol,
format.mintick)) +
416                         "\nRetracement " + str.tostring(depth * 100, "#") + "% of V leg"
417                     label.new(bar_index, adv, d == 1 ? "BUY" : "SELL", color = c,
418                         textcolor = color.white, style = d == 1 ? label.style_label_up :
label.style_label_down,

```

```

416         size = size.small, tooltip = tip)
417     box.set_border_width(s.zBox, 2)
418     s.lE := line.new(bar_index, s.entry, x2, s.entry, color = c, width = 2)
419     s.lS := line.new(bar_index, s.sl, x2, s.sl, color = slCol, style =
line.style_dashed)
420     s.lT1 := line.new(bar_index, s.tp1, x2, s.tp1, color = tgtCol, style =
line.style_dashed)
421     s.lT2 := line.new(bar_index, s.tp2, x2, s.tp2, color = tgtCol)
422     s.bS := label.new(x2, s.sl, "SL", color = color(na), textcolor = slCol,
423         style = label.style_label_left, size = size.small)
424     s.bT1 := label.new(x2, s.tp1, "TP1 " + str.tostring(s.r1, "#.#") + "R", color =
color(na),
425         textcolor = tgtCol, style = label.style_label_left, size = size.small)
426     s.bT2 := label.new(x2, s.tp2, "TP2 " + str.tostring(s.r2, "#.#") + "R", color =
color(na),
427         textcolor = tgtCol, style = label.style_label_left, size = size.small)
428
429     else if s.st == 3
430         if bar_index > s.entryBar
431             s.setTradeX(bar_index + 5)
432             bool hitSl = d * (adv - s.sl) <= 0
433             bool hitTp1 = d * (fav - s.tp1) >= 0
434             bool hitTp2 = d * (fav - s.tp2) >= 0
435             // Conservative: if SL and a target print on the same bar, the stop is assumed first
436             if hitSl
437                 if s.tp1Hit
438                     evBe := true
439                     resR := part * s.r1 + (1 - part) * d * (s.sl - s.entry) / s.risk
440                 else
441                     evSl := true
442                     resR := -1.0
443                 label.new(bar_index, adv, s.tp1Hit ? (beAfterTp1 ? "BE" : "SL after TP1") : "SL x", color
= color(na),
444                     textcolor = slCol, style = d == 1 ? label.style_label_up : label.style_label_down,
size = size.small)
445                 s.setTradeX(bar_index)
446                 s.reset()
447             else if hitTp2
448                 if not s.tp1Hit
449                     evTp1 := true
450                 evTp2 := true
451                 resR := part * s.r1 + (1 - part) * s.r2
452                 label.new(bar_index, s.tp2, "TP2 ✓", color = color(na), textcolor = tgtCol,
453                     style = d == 1 ? label.style_label_down : label.style_label_up, size = size.small)
454                 s.setTradeX(bar_index)
455                 s.reset()
456             else
457                 if hitTp1 and not s.tp1Hit
458                     s.tp1Hit := true
459                     evTp1 := true
460                     label.new(bar_index, s.tp1, "TP1 ✓", color = color(na), textcolor = tgtCol,
461                         style = d == 1 ? label.style_label_down : label.style_label_up, size =
size.small)
462                 if beAfterTp1
463                     s.sl := s.entry
464                     line.set_y1(s.lS, s.entry)
465                     line.set_y2(s.lS, s.entry)
466                     label.set_y(s.bS, s.entry)
467                     label.set_text(s.bS, "SL → BE")
468                 if bar_index - s.entryBar >= tradeMax
469                     float openR = d * (close - s.entry) / s.risk
470                     resR := s.tp1Hit ? part * s.r1 + (1 - part) * openR : openR
471                     label.new(bar_index, close, "Time exit " + str.tostring(resR, "#.###") + "R", color =
color(na),
472                         textcolor = color.gray, style = label.style_label_left, size = size.small)
473                 s.setTradeX(bar_index)
474                 s.reset()
475
476         [evArm, evEntry, evTp1, evTp2, evSl, evBe, resR]
477
478 // ----- Engine -----
479 var Setup bull = Setup.new(1)
480 var Setup bear = Setup.new(-1)
481

```

```

482 // 1 · register new liquidity pools
483 if not na(ph)
484     addPool(bslPx, bslBar, bslLn, ph, bar_index - swingLen)
485 if not na(pl)
486     addPool(sslPx, sslBar, sslLn, pl, bar_index - swingLen)
487
488 // previous-day high / low become liquidity pools at the start of each new day
489 var float dH = na
490 var float dL = na
491 var int dHB = na
492 var int dLB = na
493 if timeframe.change("D")
494     if usePdl and not na(dH)
495         addPool(bslPx, bslBar, bslLn, dH, dHB)
496         addPool(sslPx, sslBar, sslLn, dL, dLB)
497     dH := high
498     dHB := bar_index
499     dL := low
500     dLB := bar_index
501 else
502     if na(dH) or high > dH
503         dH := high
504         dHB := bar_index
505     if na(dL) or low < dL
506         dL := low
507         dLB := bar_index
508
509 // Asia session high / low become liquidity pools when the session ends
510 bool inAsia = not na(time(timeframe.period, asiaSess, kzTz))
511 var float aH = na
512 var float aL = na
513 var int aHB = na
514 var int aLB = na
515 if inAsia
516     if not inAsia[1] or na(aH)
517         aH := high
518         aHB := bar_index
519         aL := low
520         aLB := bar_index
521     else
522         if high > aH
523             aH := high
524             aHB := bar_index
525         if low < aL
526             aL := low
527             aLB := bar_index
528 else if inAsia[1] and useAsia and not na(aH)
529     addPool(bslPx, bslBar, bslLn, aH, aHB)
530     addPool(sslPx, sslBar, sslLn, aL, aLB)
531
532 // daily trade / loss limits (reset with the trading day)
533 var int dayTrades = 0
534 var int dayLosses = 0
535 if timeframe.change("D")
536     dayTrades := 0
537     dayLosses := 0
538 bool dayOk = dayTrades < maxDay and dayLosses < maxLossDay
539
540 // 2 · liquidity sweeps start (or deepen) a setup
541 [sslLv1, sslB] = scanSweeps(sslPx, sslBar, sslLn, 1)
542 [bslLv1, bslB] = scanSweeps(bslPx, bslBar, bslLn, -1)
543 if showBull and bullBias and not na(sslLv1)
544     bull.start(sslLv1, sslB)
545 if showBear and bearBias and not na(bslLv1)
546     bear.start(bslLv1, bslB)
547
548 // 3 · FVG inversions (only FVGs formed around the sweep, inverted by a displacement candle)
549 int bullMin = bull.st == 1 and bullDisp ? bull.sweepBar - fvgLook : bar_index + 1
550 int bearMin = bear.st == 1 and bearDisp ? bear.sweepBar - fvgLook : bar_index + 1
551 [bzt, bzb, bzbar] = scanInversions(bearFvgTop, bearFvgBot, bearFvgBar, 1, bullMin)
552 [szt, szb, szbar] = scanInversions(bullFvgTop, bullFvgBot, bullFvgBar, -1, bearMin)
553
554 if high < low[2] and low[2] - high >= fvgMinAtr * atr
555     addFvg(bearFvgTop, bearFvgBot, bearFvgBar, low[2], high, bar_index - 1)

```

```

556 if low > high[2] and low - high[2] >= fvgMinAtr * atr
557     addFvg(bullFvgTop, bullFvgBot, bullFvgBar, low, high[2], bar_index - 1)
558
559 // 4 · advance state machines
560 // one position at a time: a direction may not enter while the other is in a trade
561 [bArm, bEntry, bTp1, bTp2, bSl, bBe, bR] = bull.update(atr, minorLow, bzt, bzb, bzbar, bullBias, inKz and
day0k and bear.st != 3)
562 [sArm, sEntry, sTp1, sTp2, sSl, sBe, sR] = bear.update(atr, minorHigh, szt, szb, szbar, bearBias, inKz
and day0k and bull.st != 3)
563 dayTrades += (bEntry ? 1 : 0) + (sEntry ? 1 : 0)
564 dayLosses += (not na(bR) and bR < 0 ? 1 : 0) + (not na(sR) and sR < 0 ? 1 : 0)
565
566 // ----- Stats -----
567 var int    nTrades = 0
568 var int    nClosed = 0
569 var int    nWin    = 0
570 var int    nTp1    = 0
571 var int    nTp2    = 0
572 var int    nLoss    = 0
573 var float  netR     = 0.0
574 nTrades += (bEntry ? 1 : 0) + (sEntry ? 1 : 0)
575 nTp1    += (bTp1 ? 1 : 0) + (sTp1 ? 1 : 0)
576 nTp2    += (bTp2 ? 1 : 0) + (sTp2 ? 1 : 0)
577 nLoss    += (bSl ? 1 : 0) + (sSl ? 1 : 0)
578 for r in array.from(bR, sR)
579     if not na(r)
580         nClosed += 1
581         nWin    += r > 0 ? 1 : 0
582         netR     += r
583
584 statRow(table t, int row, string k, string v, color c) =>
585     table.cell(t, 0, row, k, text_color = color.silver, text_size = size.small, text_halign =
text.align_left)
586     table.cell(t, 1, row, v, text_color = c, text_size = size.small, text_halign = text.align_right)
587
588 pct(int a, int b) => b > 0 ? str.toString(100.0 * a / b, "#") + "%" : "-"
589
590 stateName(Setup s) =>
591     s.st == 0 ? "Idle" : s.st == 1 ? "Swept – waiting V/CISD/IFVG" : s.st == 2 ? "IFVG armed – waiting
retest" : "In trade"
592
593 var table stats = table.new(position.top_right, 2, 13, bgcolor = color.new(#131722, 10), border_width =
1)
594 if showStats and barstate.islast
595     table.cell(stats, 0, 0, "IFVG Sniper · " + timeframe.period + (timeframe.period == "15" ? "" : "
(tuned for 15)"),
596         text_color = color.white, text_size = size.small)
597     table.merge_cells(stats, 0, 0, 1, 0)
598     statRow(stats, 1, "Trades closed", str.toString(nClosed) + " / " + str.toString(nTrades),
color.white)
599     statRow(stats, 2, "Win rate (R > 0)", pct(nWin, nClosed), tgtCol)
600     statRow(stats, 3, "TP1 reached", pct(nTp1, nClosed), tgtCol)
601     statRow(stats, 4, "TP2 reached", pct(nTp2, nClosed), tgtCol)
602     statRow(stats, 5, "Stopped before TP1", pct(nLoss, nClosed), slCol)
603     statRow(stats, 6, "Net R", str.toString(netR, "#.##"), netR >= 0 ? tgtCol : slCol)
604     statRow(stats, 7, "Avg R / trade", nClosed > 0 ? str.toString(netR / nClosed, "#.##") : "-", netR >=
0 ? tgtCol : slCol)
605     statRow(stats, 8, "HTF bias", not useHtf ? "off" : bullBias ? "Bullish" : bearBias ? "Bearish" :
"Neutral – no trades",
606         not useHtf ? color.gray : bullBias ? bullCol : bearBias ? bearCol : color.gray)
607     statRow(stats, 9, "Killzone", not useKz ? "off" : inKz ? "Open" : "Closed", inKz ? tgtCol :
color.gray)
608     statRow(stats, 10, "Today", str.toString(dayTrades) + " trades · " + str.toString(dayLosses) + "
losses" + (day0k ? "" : " – done"),
609         day0k ? color.white : slCol)
610     statRow(stats, 11, "Bull", stateName(bull), bullCol)
611     statRow(stats, 12, "Bear", stateName(bear), bearCol)
612
613 // ----- Data window -----
614 plot(bull.st == 3 ? bull.entry : na, "Bull entry", bullCol, display = display.data_window)
615 plot(bull.st == 3 ? bull.sl : na, "Bull SL", slCol, display = display.data_window)
616 plot(bull.st == 3 ? bull.tp1 : na, "Bull TP1", tgtCol, display = display.data_window)
617 plot(bull.st == 3 ? bull.tp2 : na, "Bull TP2", tgtCol, display = display.data_window)
618 plot(bear.st == 3 ? bear.entry : na, "Bear entry", bearCol, display = display.data_window)

```

```

619 plot(bear.st == 3 ? bear.sl      : na, "Bear SL",      slCol,      display = display.data_window)
620 plot(bear.st == 3 ? bear.tp1    : na, "Bear TP1",     tgtCol,      display = display.data_window)
621 plot(bear.st == 3 ? bear.tp2    : na, "Bear TP2",     tgtCol,      display = display.data_window)
622
623 // ----- Alerts -----
624 alertcondition(bArm, "Bull IFVG armed", "IFVG Sniper: bullish sweep + V + CISD + IFVG armed on
{{ticker}} ({{interval}})")
625 alertcondition(bEntry, "Bull entry", "IFVG Sniper: BUY on {{ticker}} @ {{close}} - IFVG retest +
sweep")
626 alertcondition(bTp1, "Bull TP1", "IFVG Sniper: bullish TP1 hit on {{ticker}}")
627 alertcondition(bTp2, "Bull TP2", "IFVG Sniper: bullish TP2 hit on {{ticker}}")
628 alertcondition(bSl, "Bull stop", "IFVG Sniper: bullish stop hit on {{ticker}}")
629 alertcondition(sArm, "Bear IFVG armed", "IFVG Sniper: bearish sweep + V + CISD + IFVG armed on
{{ticker}} ({{interval}})")
630 alertcondition(sEntry, "Bear entry", "IFVG Sniper: SELL on {{ticker}} @ {{close}} - IFVG retest +
sweep")
631 alertcondition(sTp1, "Bear TP1", "IFVG Sniper: bearish TP1 hit on {{ticker}}")
632 alertcondition(sTp2, "Bear TP2", "IFVG Sniper: bearish TP2 hit on {{ticker}}")
633 alertcondition(sSl, "Bear stop", "IFVG Sniper: bearish stop hit on {{ticker}}")
634
635 if bEntry
636     alert("IFVG Sniper BUY " + syminfo.ticker + " | entry " + str.tostring(bull.entry, format.mintick) +
637         " | SL " + str.tostring(bull.sl, format.mintick) + " | TP1 " + str.tostring(bull.tp1,
format.mintick) +
638         " | TP2 " + str.tostring(bull.tp2, format.mintick), alert.freq_once_per_bar_close)
639 if sEntry
640     alert("IFVG Sniper SELL " + syminfo.ticker + " | entry " + str.tostring(bear.entry, format.mintick) +
641         " | SL " + str.tostring(bear.sl, format.mintick) + " | TP1 " + str.tostring(bear.tp1,
format.mintick) +
642         " | TP2 " + str.tostring(bear.tp2, format.mintick), alert.freq_once_per_bar_close)
643

```